C++面向对象

概念

对象（Object）是具有特殊属性(数据)和行为方式(方法)的客观存在。

类（Class）是具有相同或类似属性和方法的对象的集合。允许在已有类的基础上通过增加新特征而派生出新的类，继承关系。

类有两种成员：属性（数据成员）和行为（函数成员）

面向对象的基本特征：抽象、封装、继承和多态。

抽象：数据抽象和行为抽象。

封装：**通过接口定义**，用户只能访问对象封装界面之上的信息即外特性（对象能接收哪些消息，具有哪些处理能力），而对象的内特性（保存内部状态的私有数据和实现加工能力的算法）对用户是隐藏的。

继承：在已有类的基础上通过增加新特征而派生出新的类

多态：面向对象系统的多态性是指不同的对象收到相同消息时产生多种不同的行为方式。即，”一个接口，多种形态“。

C++语言支持两种多态性，即**编译时多态性**和**运行时多态性**。分别由重载和虚函数来实现。

引用

1.常量引用

函数中通过利用常量引用来防止误操作修改实参

const 常量类型 常量名=常量值

常量必须要给他赋初值

> 常量引用：在前面加上const

```
void showValue(const int& v)
{
    cout << v << endl;
}

int main()
{
    //int& ref = 10; 错误，因为引用需要一个合法的内存空间；
    //修改为下面等同于：int temp=10;const int&ref=temp;
    const int& ref = 10;
    int a = 10;
    showValue(a);
    return 0;
}
```

在另一连接文件中引用const常量

```
extern const int i;       //合法
extern const int i=10;    //非法，常量不可以被再次赋值
```

#define是C语言中用来定义宏常量，const定义常量比#define定义常量有更多的优点。
（1）const常量有数据类型，而宏常量没有数据类型。编译器可以对前者进行类型安全检查。而对后者只进行字符替换，没有类型安全检查，并且在字符替换可能会产生意料不到的错误。
（2）有些集成化的调试工具可以对const常量进行调试，但是不能对宏常量进行调试。

可以利用预处理指令#define来定义宏，而使用#undef删除由#define定义的宏，使之不再起作用。

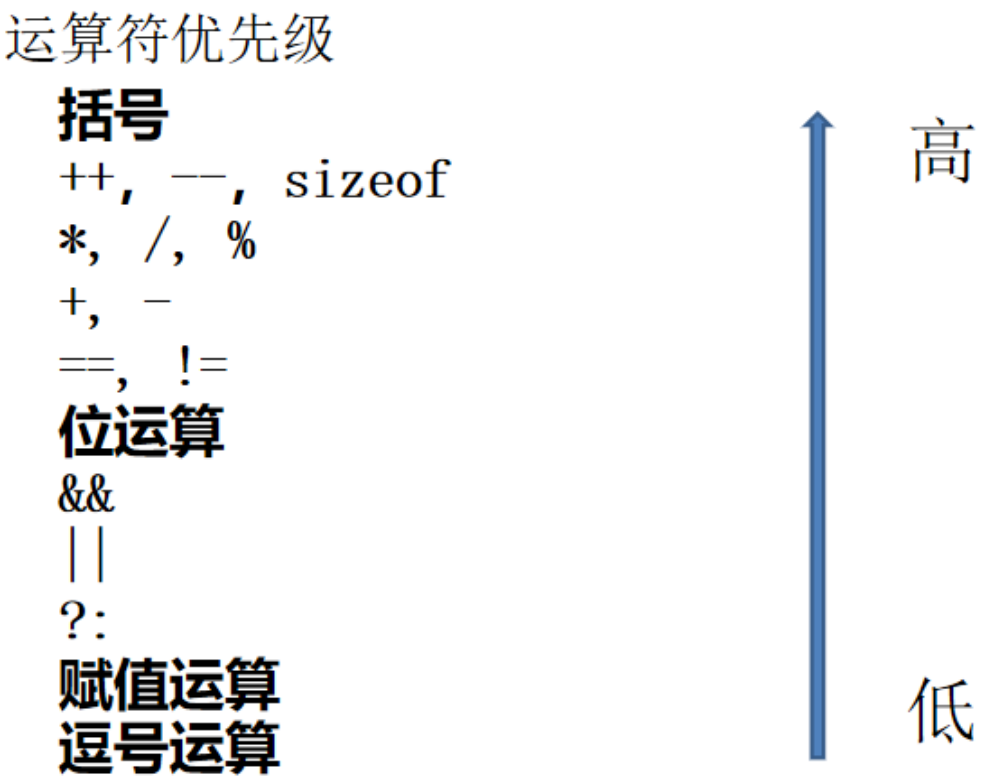
2.存储类型

auto
属于一时性存储，其存储空间可以被若干变量多次覆盖使用。
register
存放在通用寄存器中。
extern
在所有函数和程序段中都可引用。
static
在内存中是以固定地址存放的，在整个程序运行期间都有效。

sizeof运算符

“类型名”所指定的类型或“表达式”的结果类型所占的字节数。

按位与（&）按位或（|）按位异或（^）按位取反（~）



3.指针

字符串处理函数：strcat（连接），strcpy（复制），strcmp（比较），strlen（求长度）， strlwr(转换为小写），strupr（转换为大写）

指针常量和常量指针

指针常量

在指针定义语句的指针名前加const, 表示指针本身是常量。int a; int* const p=&a; //定义时必须初始化，即p是指针值p不可以修改，指针指向的内容可以修改。常量，不可以作为左值进行运算，*p可以修改

常量指针（指向常量的指针）

在指针的定义类型前加const，表示指向的对象是常量。如const int *pi 或 int const *pi; 均可。以上定义表明*pi是常量，不能将*pi作为左值进行操作。定义指向常量的指针时，定义时不必须先初始化。

指针函数与函数指针

指针函数：返回指针值的函数。

定义格式：`类型修饰符* 函数名（形参表）{函数体}`

例：int* f(int a);

函数指针：指向函数地址（程序区）的指针。与函数名等价的指针。函数名是指向函数的指针常量。

函数指针定义：

若函数原型为：类型 函数名(形参表);

则相应函数指针定义为：

类型 (*变量名*) (*形参表*)；// ()*的优先级大于*

```
  char fun(int);        char (*p)(int );
例：int f1(int n);
    int (*pf1)(int n);
    pf1=f1;//pf1是与f1等价的函数指针
```

指针函数用来申请空间或者查找数据;
函数指针用来临时制定参与运算的函数

指针函数用来申请空间或者查找数据;
函数指针用来临时制定参与运算的函数

枚举类型

`enum  枚举类型名  {变量值列表};`

联合体

```
union uarea
{  char   c_data;
   short  s_data;
   long   l_data;
}
```

所有的成员共享一个空间,同一时间只有一个成员是有效的,Union的大小是其最大的成员

对第一个成员做初始化

```
typedef union {
    int i;
    char ch[sizeof(int)];
} CHI;

int main() {

    CHI chi;
    chi.i = 1234; // 十六进制 0x04D2
    for(int i=0; i<sizeof(int); i++)
    {
        printf("%02hhx", chi.ch[i]);
    }
    // 若输出 D2040000, 则为小端方式
    // 若输出 000004D2, 则为大端方式
    printf("\n");
```

函数

函数原型作用域是C++程序中最小的作用域

1.函数默认参数

> **语法:** 返回值类型 函数名（数据类型 参数=默认值）{}

```
//如果自己有传入值，就可以用自己的数据，没有就用默认值
int func(int a, int b=10, int c=10)
{
    return a + b + c;
}

int main()
{
    cout << func(10,20,0) << endl;
    return 0;
}
```

注意:

1.如果某个位置已经有了默认参数，那么该位置向后的所有都必须有默认值。

EX: ~~int func(int a,int b=10,int c,int d)~~

2.如果函数声明有默认参数，那么函数实现就不能有默认参数，声明和实现只能二选一

EX：~~int func(int a=10,int b=10);~~

~~int func(int a=10,int b=10) {}~~

2.函数占位参数

> **语法：** 返回值类型 函数名 （数据类型）{}

```
//占位参数也可以有默认值
int func(int a, int )
{
    return a ;
}

int main()
{
    //占位参数必须填补
    cout << func(10,20) << endl;
    return 0;
}
```

3.函数重载

可以让函数名重复，提高复用性

需要满足的条件：

1.在同一个作用域。

2.函数名相同。

3.函数参数类型不同，或者个数不同，或者顺序不同

注意：函数的范围值不能作为重载条件

```
void func()
{
    cout << "func!!" << endl;
}

void func(int a)
{
    cout << "func>" << endl;
}

void func(double b)
{
    cout << "func>" << endl;
}

void func(int a, double b)
{
    cout << "funccc" << endl;
}

int main()
{
    func();
    func(10);
    func(11.1);
    func(10, 11.1);
    return 0;
}
```

引用作为重载的条件

```
void func(int &a)//int &a=10;不合法
{
    cout << "func!!" << endl;
}

void func(const int& a)
{
    cout << "func>" << endl;
}

int main()
{
    int a = 10;
    func(a);//调用上面的代码
    func(10);//调用下面的代码
    return 0;
}
```

函数重载碰见默认参数

```
void func(int a)
{
    cout << "func!!" << endl;
}

void func(int a,int b=10)
{
    cout << "func>" << endl;
}

int main()
{
    func(10);//报错,碰见默认参数会碰见二义性
    func(10,10);//传两个时不会有问题
    return 0;
}
```

类和对象

c++面向对象的三大特性：封装，继承，多态

万事万物都有对象，对象上有其属性和行为

1.封装

- 封装的意义
    - 将属性和行为作为一个整体
    - 将属性和行为加以权限控制

> **语法：**class 类名{ 访问权限：属性/行为}

```
#include<iostream>
using namespace std;

const double PI = 3.14;
```

```
class Circle
{
    //访问权限
    //公共
public:
    //属性
    int r;
    double calculate()
    {
        return 2 * PI * r;
    }
};

int main()
{
    Circle C;//实例化一个对象
    C.r = 10;
    cout << "周长：" << C.calculate() << endl;
    return 0;
}
```

类中的属性和行为 统一称为成员

属性： 成员属性 成员变量

行为： 成员函数 成员方法

- 访问权限
  - public 公共权限（成员类内可以访问，类外可以访问）定义了类的外部接口
  - protect 保护权限（成员类内可以访问，类外不可以访问，儿子可以访问父亲中的保护内容）
  - private 私有权限（成员类内可以访问，类外可不以访问，儿子不可以访问父亲中的私有内容）
- struct与class区别
  - struct默认公有
  - class默认私有

```
class p1
{
    string a;//默认私有
};

struct p2
{
    string b;//默认公有
};
```

2.对象的初始化和清理

2.1构造函数和析构函数

如果我们不提供构造和析构，编译器会自动提供，编译器提供的构造函数和析构函数是空实现

构造函数： `类名（）{}`

主要用于在构造对象时为对象的成员属性赋值，构造函数由编译器自动调用，无需手动调用，没有返回值且只会调用一次，**，可以有参数可以发生重载**

析构函数： `~类名（）{}`

主要用于对象销毁前系统自动调用，没有返回值且只会调用一次，**不能有参数，不可以发生重载**

```
#include<iostream>
#include<string>
using namespace std;

class Person
{
public:
    //构造函数：
    Person()
    {
        cout << "构造函数的调用" << endl;
    }

    //析构函数；
    ~Person()
    {
        cout << "析构函数的调用" << endl;
    }
};

void test()
{
    Person p;//局部变量存储在栈上，test执行完之后自动释放
}

int main()
{
    test();
    //Person P;
    return 0;
}
```

- **构造函数的分类和调用**

按参数分类：有参构造和无参构造

按类型分类：普通构造函数和拷贝构造函数

```
class Person
{
private:
    int age;
public:
    //构造函数：
    Person()
    {
        cout << "构造函数无参的调用" << endl;
    }
    Person(int a)
    {
        age = a;
        cout << "构造函数有参的调用" << endl;
    }
```

```
    //拷贝构造函数
    Person(const Person& p)
    {
        //将传入的人身上的所有属性都拷贝到我身上
        age = p.age;
    }
};

void test()
{
    //调用
    //括号法
    Person p1;//无参
    Person p2(5);//有参
    Person p3(p2);//拷贝

    //！！注意不要带括号(),因为下面这行代码，编译器会认为是一个函数的声明
    Person p();//类似与void p();

    //显示法
    Person PP1;
    Person PP2 = Person(10);
    Person PP3 = Person(PP2);

    Person(10);//匿名对象，当前执行结束后，系统会立马回收掉匿名对象

    //不要利用拷贝函数，初始化匿名对象
    //EX：Person（PP3）;编译器会以为你在对象声明

    //隐式调用法
    Person p4 = 10;//==Person p4=Person(10);
    Person p5 = p4;
}
```

- 构造函数调用规则

  一般情况下，c++编译器会提供默认无参构造函数，有参构造函数和拷贝构造函数，若用户定义有参构造函数，便不会提供无参，但会提供拷贝。如果用户定义拷贝，不会再提供其他所有函数。

2.2拷贝构造函数

- 拷贝构造函数的调用时机
  - 使用一个已经创建完成的对象来初始化一个新对象
  - 值传递的方式给函数参数传值
  - 以值方式返回局部对象

```
//方式1：
void test()
{
    Person p1(10);
    Person p2(p1);
}

//方式2：
void doWork(Person p)
{
```

```
}

void test01()
{
    Person p;
    doWork(p);
}

//方式3:
Person doWork2()
{
    Person p1;
    return p1;
}
```

- **浅拷贝和深拷贝**

浅拷贝：简单的赋值拷贝，问题：堆区的内存重复释放

深拷贝：在堆区中重新申请空间，进行拷贝操作

```
private:
    int age;
    int* height;
public:
    //构造函数:
    Person()
    {
        age = 10;
        height = new int(11);
    }
    Person(int a,int h)
    {
        age = a;
        height = new int(h);//创建在堆区
    }
    //自己实现拷贝构造函数 解决浅拷贝带来的问题
    Person (const Person &p)
   {
        age=p.age;
        //height=p.height;编译器默认实现的拷贝函数代码
        //深拷贝操作
        height=new int(*p.height);
   }
    ~Person()
    {
        //堆区的代码需要手动释放
        if (height != NULL)
        {
            delete height;
            height = NULL;
        }
        cout << "析构函数的调用" << endl;
    }
```

2.3初始化列表

```
class Person
{
private:
    int age;
    int height;
    int weight;
public:
    Person(int a, int b, int c) :age(a), height(b), weight(c)
    {

    }
};

int main()
{
    Person p(10,20,30);
    return 0;
}
```

2.4类对象作为类成员

c++类中的成员可以是另一个类的对象，我们称该成员为对象成员

```
class A{}
class B
{
    A a;
}
```

当其他类对象作为本类成员，构造时先构造类对象，再构造自身，析构的顺序与构造相反。

```
class Phone
{
public:
    string name;
    Phone(string phone)
    {
        name = phone;
        cout << "Phone构造函数调用" << endl;
    }
    ~Phone()
    {
        cout << "Phone析构函数调用" << endl;
    }
};

class Person
{
private:
    int age;
    int height;
    Phone weight;
public:
    Person(int a, int b, string c) :age(a), height(b), weight(c)
```

```
    {
        cout << "Person构造函数调用" << endl;
    }
    ~Person()
    {
        cout << "Person析构函数调用" << endl;
    }
};
```

2.5静态成员

关键字：`static`

静态成员变量

- 所有对象共享同一份数据
- 编译阶段分配内存
- 类内声明，类外初始化

静态成员变量，不属于某个对象上，所有对象都共享同一份数据

```
class Person
{
public:
    static int A_a;
//静态成员变量也具有访问权限，A_a可以在类外访问，A_b不可以在类外访问
private:
    static int A_b;
};

int Person::A_a=100;
int Person::A_b=100;

int main()
{
    Person p1;
    cout << p1.A_a << endl;//100
    Person p2;
    p2.A_a = 200;//更改p2的A_a,p1也会变化，因为他们共同用一个数据
    cout << p1.A_a << endl;//200
    //cout << p1.A_b << endl;报错
    return 0;
}
```

有两种访问方式：

- 通过对象进行访问

  ```
  cout << p1.A_a << endl;
  ```

- 通过类名进行访问

  ```
  cout << Person::A_a << endl;
  ```

静态成员函数

```
```

```
class Person
{
public:
    static int A_a;
    int a;
    static void func()
    {
        A_a = 10;
        //a = 10;
        //静态成员函数无法访问非静态成员
    }
//静态成员函数也具有访问权限，func()可以在类外访问，func1()不可以在类外访问
private:
    static void func1()
    {
        A_a = 11;
    }
};

int Person::A_a=100;
```

- 通过对象访问

```
Person p1;
p1.func();
```

- 通过类名访问

```
Person::func();
```

3.C++对象模型和this指针

- **成员变量和成员函数分开存储**

空对象所占内存为：1

c++编译器会给每一个空对象也分配一个字节空间，是为了区分空对象占内存的位置

每个空对象也应该有一个独一无二的内存地址

```
class Person
{

};

int main()
{
    Person p;
    cout << sizeof(p) << endl;//1
    return 0;
}
```

非空对象

```
class Person
```

```
{
public:
    int a;//在类对象上，输出会变成4
    static int b;//不在类对象上，输出不会发生改变
    void func()//不在类对象上，输出不会发生改变
    {

    }
    static void func1()//不在类对象上，输出不会发生改变
    {

    }
};

int Person::b = 1;

int main()
{
    Person p;
    cout << sizeof(p) << endl;
    return 0;
}
```

- **this指针**：指向被调用的成员函数所属的对象
    - 解决名称冲突

```
class Person
{
public:
    int age;
    Person(int age)
    {
        this->age = age;
    }
};
```

    - 返回对象本身用*this

```
class Person
{
public:
    int age;
    Person(int a) :age(a)
    {

    }
    Person& personAddAge(Person p)
    {
        this->age += p.age;
        return *this;
        //this是指向对象的指针
        //返回执行以上任务结束之后的*this指向的对象本体
    }
    Person Paa(Person p)//没有用对象本体加，相当于用复制体加，所有结果不会叠加
    {
        this->age = p.age;
```

```
        return *this;
    }
};

int main()
{
    Person p1(10);
    Person p2(10);
    Person p3(10);
    //链式编程
    p2.personAddAge(p1).personAddAge(p1).personAddAge(p1);
    cout << p2.age << endl;
    //结果为40
    p3.Paa(p1).Paa(p1).Paa(p1);
    cout << p3.age << endl;
    //结果为20
    return 0;
}
```

空指针调用成员函数

```
class Person
{
public:
    int age;
    Person(int a) :age(a)
    {

    }
    void show()
    {
        cout << "1111" << endl;
    }
    void show2()
    {
        //改进方法
        if (this == NULL)
        {
            return;
        }
        //报错原因是:在下面代码中age=this->age,但是传入的指针是NULL
        cout << "age:" << age << endl;
    }
};

int main()
{
    Person* p = NULL;
    p->show();
    p->show2();
    return 0;
}
```

- **const修饰成员函数**

  常函数：成员函数后加const，不能修改成员属性

  成员属性声明时加关键字mutable后，在常函数中可以修改

```
class Person
{
public:
    int m_A;
    mutable int m_B;
    void showPerson() const
    {
        //加上const后不能修改指针指向的值
        // const Person;
        //m_A=100==this->m_A = 100;
        //不能修改指针指向
        //Person *const this;
        //this = NULL;
        this->m_B = 10;//特殊变量可以修改
        m_B = 100;
    }
};
```

常对象：声明对象前加const，只能调用常函数

```
const Person p;
p.showPerson();
//p.m_A = 10;
p.m_B = 10;
```

4.友元

关键字：`friend`

目的：让一个函数或者类访问另一个类中的私有成员

4.1全局函数做友元

> 关键：friend void goodGay(Building* building);

```
class Building
{
    friend void goodGay(Building* building);
public:
    string m_settingRoom;
    Building()
    {
        m_settingRoom = "客厅";
        m_bedRoom = "卧室";
    }
private:
    string m_bedRoom;
};

void goodGay(Building *building)
{
    cout << building->m_settingRoom << endl;
    cout << building->m_bedRoom << endl;
    //需要在类中声明友元才能使用这个私有变量
}

int main()
```

```
{
    Building b;
    goodGay(&b);
    return 0;
}
```

4.2类做友元

> 关键：friend GoodGay;
>
> void GoodGay::visit()
>
> GoodGay::GoodGay()

```
class Building;
class GoodGay
{
public:
    Building* building;
public:
    GoodGay();
    void visit();
};

class Building
{
    friend GoodGay;
public:
    Building();
public:
    string settingRoom;
private:
    string bedRoom;
};

//类外声明构造函数
GoodGay::GoodGay()
{
    building = new Building;
}

Building::Building()
{
    settingRoom = "settingroom";
    bedRoom = "bedroom";
}

void GoodGay::visit()
{
    cout << building->settingRoom << endl;
    cout << building->bedRoom << endl;
}
```

4.3 成员函数做友元

> 关键：friend void GoodGay::visit();

```
class GoodGay
{
public:
    Building* building;
public:
    GoodGay();
    void visit();//让其可以访问Building中的私有成员
    void visit2();//让其不可以访问Building中的私有成员
};

class Building
{
    friend void GoodGay::visit();
public:
    Building();
public:
    string settingRoom;
private:
    string bedRoom;
};
```

5.运算符重载

对已有的运算符重新进行定义，赋予其另一种功能以适应不同的数据类型

5.1加号

```
//成员函数重载加号
Person operator+(Person& p)
{
    Person p3;
    p3.m_A = p.m_A + this->m_A;
    p3.m_B = p.m_B + this->m_B;
    return p3;
}
int main()
{
    Person p1;
    Person p2;
    Person p3;
    p1.m_A = 10;
    p1.m_B = 20;
    p2.m_A = 1;
    p2.m_B = 2;
    p3 = p1 + p2;
    //本质p3=p1.operator+(p2);
    return 0;
}
```

```
//全局函数实现加号重载
Person operator+(Person& p1, Person& p2)
{
```

```
    Person p3;
    p3.m_A = p1.m_A + p2.m_A;
    p3.m_B = p1.m_B + p2.m_B;
    return p3;
}
int main()
{
    Person p1;
    Person p2;
    Person p3;
    p1.m_A = 10;
    p1.m_B = 20;
    p2.m_A = 1;
    p2.m_B = 2;
    p3 = p1 + p2;
    //本质p3=operator+(p1+p2);
    return 0;
}
```

运算符重载也可以发生函数重载

```
//全局函数实现加号重载
//Person operator+(Person& p1, int n)
//{
//  Person p3;
//  p3.m_A = p1.m_A + n;
//  p3.m_B = p1.m_B + n;
//  return p3;
//}
```

5.2左移运算符

不能利用成员函数重载`<<`，无法实现。只能使用全局函数重载

输出流：`ostream`

```
void operator<<(ostream& cout, Person& p)//有无&都可以，加上&只是为了节约内存空间
{
    cout << "m_A: " << p.m_A << " m_B: " << p.m_B;
}

int main()
{
    Person p;
    cout << p;//返回值是void后面不能加上<<
    return 0;
}
```

```
ostream& operator<<(ostream& cout, Person& p)
{
    cout << "m_A: " << p.m_A << " m_B: " << p.m_B;
    return cout;
}

int main()
{
    Person p;
    cout << p << " hello world" << endl;//链式输出
    return 0;
}
```

5.3递增运算符

```
class Person
{
    friend ostream& operator<<(ostream& cout, Person p);
public:
    Person() {};
    //前置++
    Person& operator++()
    {
        m_A++;
        m_B++;
        return *this;
    }
    //后置++
    Person operator++(int)//int 作为占位符，用于区分前置和后置
    {
        Person p;
        p.m_A = this->m_A;
        p.m_B = this->m_B;
        m_A++;
        m_B++;
        return p;
    }
private:
    int m_A = 10;
    int m_B = 10;
};

ostream& operator<<(ostream& cout, Person p)
{
    cout << "m_A: " << p.m_A << " m_B: " << p.m_B;
    return cout;
}


int main()
{
    Person p;
    cout << ++p << endl;
    cout << p << endl;
    cout << p++ << endl;
    cout << p << endl;
    return 0;
```

```
}
```

5.4赋值运算符

赋值运算符`operator=`，对属性进行拷贝

```
#include<iostream>
#include<string>
using namespace std;

class Person
{
    friend ostream& operator<<(ostream& cout, Person& p);
public:
    Person(int a)
    {
        m_A = new int(a);
    }
    ~Person()
    {
        if (m_A != NULL)
        {
            delete m_A;
            m_A = NULL;
        }
    }
    Person& operator=(Person &p)
    {
        /*m_A = p.m_A;
        m_B = p.m_B;*/
        //以上是编译器自带的浅拷贝，后续释放空间程序会崩溃
        if (m_A != NULL)//先判断是否有属性在堆区
        {
            delete m_A;
            m_A = NULL;
        }
        m_A = new int(*p.m_A);

        return* this;
    }
private:
    int* m_A;
};

ostream& operator<<(ostream& cout, Person& p)
{
    cout << *p.m_A;
    return cout;
}

int main()
{
    Person p(10);
    Person p1(20);
    Person p3(30);
    p3 = p1 = p;
    cout << p << " " << p1 << " " << p3 << endl;
```

```
    return 0;
}
```

5.5关系运算符

```
class Person
{
public:
    string name;
    int age;
    Person(string n, int a) :name(n), age(a)
    {

    }
    bool operator==(Person& p)
    {
        if (this->name == p.name && this->age == p.age)
        {
            return true;
        }
        else
        {
            return false;
        }
    }
    bool operator != (Person& p)
    {
        if (this->name != p.name || this->age != p.age)
        {
            return true;
        }
        else
        {
            return false;
        }
    }
};
```

5.6函数调用运算符

因为重载后使用的方式很像函数的调用，所以称为仿函数

```
class Person
{
public:
    string operator()(string name)
    {
        return name;
    }
};



int main()
{
    //匿名对象
    cout << Person()("1111") << endl;
```

```
    return 0;
}
```

6.继承

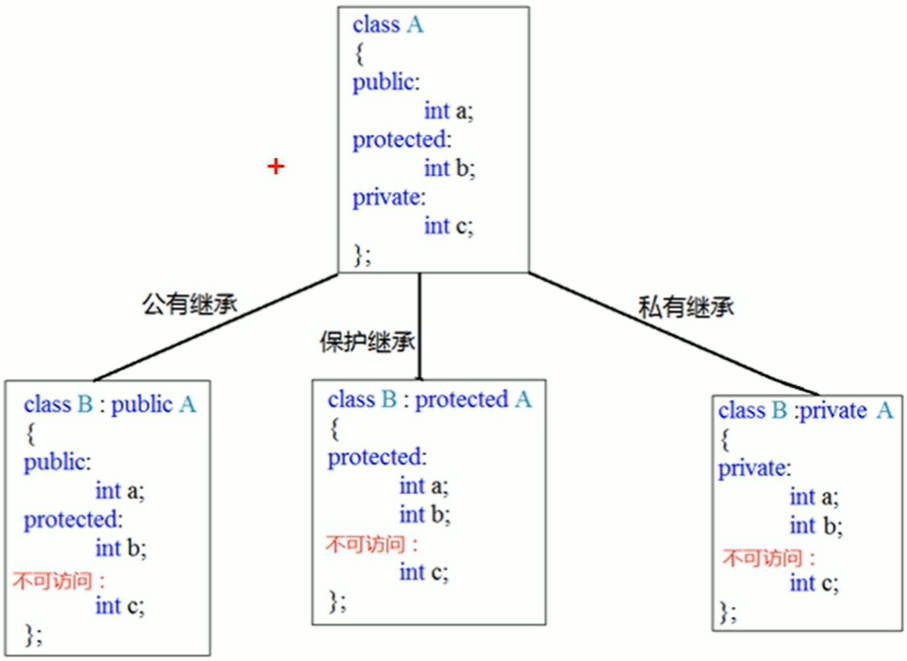
> 语法：class 子类（派生类）：继承方法 父类（基类）

A继承B：`class A : public B`

好处：减少重复代码

6.1继承方式

- 公共继承 `Public`
- 保护继承 `protected`
- 私有继承 `private`




```
class Base1
{
public:
    int a;
protected:
    int b;
private:
    int c;
};

class Son1 :public Base1
{
public:
    void func()
    {
        a = 10;
        b = 10;//不会变为公有，还是保护权限
        //c = 10;私有无法访问
```

```
    }
};

class Son2 :protected Base1
{
public:
    void func()
    {
        a = 10;//变为保护
        b = 10;
        //c = 10;私有无法访问
    }
};

class Son3 :private Base1
{
public:
    void func()
    {
        a = 10;//变为私有
        b = 10;//变为私有
        //c = 10;私有无法访问
    }
};

int main()
{
    Son1 s1;
    s1.a = 20;
    //s1.b = 10;保护权限类外无法访问
    Son2 s2;
    Son3 s3;
    //s2.a = 10;保护权限类外无法访问
    //s3.a = 20;私有权限类外无法访问
    return 0;
}
```

6.2继承中的对象模型

父类中所以非静态属性都会被子类继承下来

父类中的私有属性，继承后被编译器隐藏，所以访问不到

利用开发人员命令题诗工具查看对象模型：

跳转盘符

跳转文件路径`cd`具体路径下

查看命名

`cl /d1 reportSingleClassLayout类名 文件名`

6.3继承中的构造和析构

顺序如下：

先构造父类，再构造子类；析构的顺序与构造的顺序相反。

6.5继承中同名处理

```
class Base1
{
public:
    int a = 100;
    void func()
    {
        cout << "Base!" << endl;
    }
    void func(int aa)
    {
        cout << "Base1!" << endl;
    }
protected:
    int b;
private:
    int c;
};

class Son1 :public Base1
{
public:
    int a = 200;
    void func()
    {
        cout << "Son1!" << endl;
    }
};

int main()
{
    Son1 s;
    cout << s.a << endl;//200;
    //如果需要通过子类对象 访问到父类同名成员需要添加作用域
    cout << s.Base1::a << endl;//100;
    s.func();
    //当子类与父类拥有同名的成员函数，子类会隐藏父类中所以版本的同名成员数
    //如果想访问父类在被隐藏的同名函数，需要添加父类的作用域
    s.Base1::func();
    s.Base1::func(100);
    return 0;
}
```

- 静态成员同名

```
class Base1
{
public:
    static int m_A ;
    static void func()
    {
        cout << "1" << endl;
    }
};

int Base1::m_A = 100;
```

```
class Son1 :public Base1
{
public:
    static int m_A;
    static void func()
    {
        cout << "2" << endl;
    }
};

int Son1::m_A = 200;

int main()
{
    Son1 s;
    cout << s.m_A << endl;
    cout << s.Base1::m_A << endl;
    cout << Son1::m_A << endl;
    cout << Son1::Base1::m_A << endl;
    s.func();
    s.Base1::func();
    Son1::func();
    Son1::Base1::func();
    return 0;
}
```

6.6多继承语法

语法`class 子类：继承方式 父类1，继承方式 父类2`

```
class Base1
{
public:
    int m_A;
};

class Base2
{
public:
    int m_A;
};

class Son1 :public Base1,public Base2
{
    int m_A;
};

int main()
{
    Son1 s;
    //若父类出现重名
    s.Base1::m_A = 20;
    s.Base2::m_A = 30;
    return 0;
}
```

6.7菱形继承---钻石继承

使用虚继承解决这个问题: `virtual`

```
class animal
{
public:
    int age;
};

class Sheep :virtual public animal {};
class Tuo :virtual public animal {};
class SheepTuo :public Sheep, public Tuo {};

int main()
{
    //如果不使用虚继承，sheeptuo类会继承两个age，出错
    //声明从哪一个类的路径继承之后不会报错，但是内存开销大
    SheepTuo st;
    Sheep s;
    Tuo t;
    s.age = 10;
    t.age = 20;
    st.Sheep::age = 200;
    st.Tuo::age = 300;
    st.age = 400;
    //使用虚继承后无论是通过那一类进行继承，所得结果都一样，但是不会改变父类的值
    cout << st.Sheep::age << endl;
    cout << st.Tuo::age << endl;
    cout << st.age << endl;
    cout << s.age << endl;
    cout << t.age << endl;
    return 0;
}
```

7.多态

7.1多态的概念

分类:

- 静态多态：函数重载、运算符重载、复用函数名

  函数地址早绑定—编译阶段确定函数地址

- 动态多态：派生类、虚函数实现

  函数地址晚绑定—运行阶段确定函数地址

```
#include<iostream>
#include<string.h>
using namespace std;

class Animal
{
public:
    //加上virtual关键字，变成虚函数，编译的时候不能确定函数调用
    virtual void speak()
    {
```

```
        cout << "nyanya" << endl;
    }
};

class Cat :public Animal
{
public:
    //重写虚函数
    void speak()
    {
        cout << "miaomiao" << endl;
    }
};

class Dog :public Animal
{
public:
    void speak()
    {
        cout << "wangwang" << endl;
    }
};

//引用进行调用
void speak(Animal &animal)
{
    animal.speak();
}

int main()
{
    Cat cat;
    Dog dog;
    cat.speak();
    dog.speak();
    //Animal *a;
    //a=new Dog;
    //a = &cat;
    //a->speak();
    //a = &dog;
    //a->speak();
    return 0;
}
```

多态满足条件：

- 有继承关系
- 子类重写父类中的虚函数

多态使用条件：

- 父类指针或引用指向子类对象

重写：函数返回值类型，函数名，返回值列表完全一直叫做重写

好处：

- 组织结构清晰
- 可读性强

- 对于前期和后期的可读性强

7.2纯虚函数和抽象类

纯虚函数语法：`virtual 返回值类型 函数名 （参数列表） = 0;`

当父类中虚函数的实现是毫无意义的，主要通过调用子类重写的内容时，可以将虚函数改写为纯虚函数

当类中有了抽象类，这个类也被称为抽象类

抽象类特点：

- 无法实例化对象
- 子类必须重写抽象类中的纯虚函数，否则也属于抽象类

```
#include<iostream>
using namespace std;

class Base
{
public:
    virtual void func() = 0;
};

class Son:public Base
{
public:
    void func()
    {
        cout << "111" << endl;
    }
};

int main()
{
    //Base b;//抽象类无法实例化
    Base* base = NULL;
    base = new Son;
    base->func();
    delete base;
    return 0;
}
```

7.3虚析构和纯虚析构

多态使用时候，如果有子类中属性开辟到堆区，那么父类指针在释放时无法调用子类的析构函数；

解决方法：将父类中的析构函数改成虚析构和纯虚析构

纯虚析构属于抽象类，无法实例化对象

`virtual ~类名(){};`

`virtual ~类名()=0;` //类外需要实现

```
class Base
{
public:
    //纯虚析构也需要在内外实现
```

```
    virtual ~Base() {} = 0;

    /*virtual ~Base()
    {
        cout << "Base析构" << endl;
    }*/

    virtual void func() = 0;
};

Base::~Base()
{
    cout << "纯虚析构实现" << endl;
}
```

8.模板

建立通用的模板，大大提高**复用性**

以下不包含分文件编写和另外封装

8.1函数模板

```
template<typename T>//typename 可以替换成class
函数声明定义
```

```
template<typename T>
void mySwap(T& a, T& b)
{
    T temp = a;
    a = b;
    b = temp;
}

int main()
{
    int a = 10;
    int b = 20;
    //自动类型推导
    mySwap(a, b);

    //显示指定类型
    mySwap<int>(a, b);
}
```

- 注意事项

1.自动类型推导，必须推导出一致的数据类型T才可以使用

```
template<typename T>
void mySwap(T& a, T& b)
{
    T temp = a;
    a = b;
    b = temp;
}

int a = 10;
int b = 20;
char c = 'c';
//mySwap(a, c);出错
```

2.模板必须要确定出T的数据类型，才可以使用

```
template<typename T>
void func()
{
    cout << "1" << endl;
}

//func();出错
func<int>();
```

- 普通函数和函数模板的区别

  1.普通函数**可以发生**隐式类型转换

  2.函数模板用自动类型推导**不会发生**隐式类型转换

  3.函数模板用指定显示类型**可以发生**隐式类型转换

- 普通函数和函数模板的调用规则

  1.普通函数和函数模板都可以调用，优先调用普通函数

  2.可以通过空模板的参数列表强行调用函数模板

  3.函数模板可以发生函数重载

  4.如果函数模板可以产生更好的匹配，优先调用函数模板

```
#include<iostream>
#include<math.h>
using namespace std;

template<typename T>
void mySwap(T& a, T& b)
{
    T temp = a;
    a = b;
    b = temp;
}

template<typename T>
void mySwap(T& a, T& b,T &c)
{
    T temp = a;
    a = b;
    b = temp;
```

```
}

void mySwap(int &a,int &b)
{
    int temp = a;
    a = b;
    b = temp;
}

int main()
{
    int a = 10;
    int b = 20;
    int c = 100;

    int x = 'x';
    int y = 'y';

    //优先调用普通函数，当普通函数没有实现的时候也不会调用模板而是会报错
    mySwap(a, b);
    //可以通过空模板的参数列表强行调用函数模板
    mySwap<>(a, b);
    //函数重载
    mySwap(a, b, c);
    //如果函数模板可以产生更好的匹配，优先调用函数模板
    mySwap(x, y);
}
```

- 模板的局限性

  利用具体化的模板可以解决自定义类型的通用化

```
class Person
{
public:
    int a;
    string b;
};

template<typename T>
void mySwap(T& a, T& b)
{
    T temp = a;
    a = b;
    b = temp;
}

//利用具体化Person的版本实现代码，具体化优先调用
template<>void mySwap(Person& p1, Person& p2)
{
    p1.a = p2.a;
    p1.b = p2.b;
}
```

8.2类模板

```
#include<iostream>
#include<string>
#include<math.h>
using namespace std;

template<class nameType,class ageType>
class person
{
public:
    person(string name, int age)
    {
        this->name = name;
        this->age = age;
    }

    string name;
    int age;
};

int main()
{
    person<string, int> p1("aaa", 18);
    return 0;
}
```

8.3区别

- 类模板没有自动推导类型
- 类模板在模板参数列表可以有默认参数

```
template<class nameType,class ageType=int>
class person
{
public:
    person(string name, int age)
    {
        this->name = name;
        this->age = age;
    }

    string name;
    int age;
};

int main()
{
    person<string> p1("aaa", 18);
    return 0;
}
```

- 普通类中的成员函数一开始就可以创建
- 类模板中的成员函数在调用的时候创建

8.4类模板做函数参数

一共有三种传入方式

1.指定传入的类型

2.参数模板化

3.整个类模板化

```
#include<iostream>
#include<string>

using namespace std;

template<class T1,class T2>
class Person
{
public:
    Person(T1 name, T2 age)
    {
        this->name = name;
        this->age = age;
    }

    void shouData()
    {
        cout << name << " " << age << endl;
    }

    string name;
    int age;
};

//指定参数传入
void printPerson1(Person<string, int>& p)
{
    p.shouData();
}

//参数模板化
template<class T1,class T2>
void printPerson2(Person<T1, T2>& p)
{
    p.shouData();
}

//整个类模板化
template<class T>
void printPerson3(T& p)
{
    p.shouData();
}

int main()
{
    Person<string, int>p("aaa", 18);
    printPerson1(p);
```

```
    printPerson2(p);
    printPerson3(p);
    return 0;
}
```

8.5类模板与继承

- 当子类继承的父类是一个类模板的时，子类在声明的时候，要指定出父类中的类型
- 如果不指定，编译器无法给子类分配内存
- 如果像灵活指定出父类中T的类型，子类也需变为类模板

```
template<class T>
class Base
{
public:
    T m;
};

class Son1 :public Base<int>
{

};

template<class T1,class T2>
class Son2 :public Base<T2>
{
public:
    T1 n;
};

int main()
{
    Son1 s1;
    Son2<int, char> s2;
    return 0;
}
```

8.6类模板成员函数类外实现

```
#include<iostream>
using namespace std;

template<class T>
class Base
{
public:
    Base();

    void func();
};

template<class T>
Base<T>::Base()
{
    cout << "构造函数" << endl;
}
```

```
template<class T>
void Base<T>::func()
{
    cout << "成员函数" << endl;
}
```

需要加上模板的参数列表

8.7类模板与友元

```
#include<iostream>
using namespace std;

template<class T>class Base;
//如果声明了函数模板，可以把实现写到后面，不然就要将时下媒体写在前面让编译器提前看到
template<class T>
void printBase1(Base<T>& b)
{
    cout << "111" << endl;
}

template<class T>
class Base
{
    //全局函数配合友元类内实现
    friend void printBase(Base<T>& b)
    {
        cout << "12345" << endl;
    }

    //全局函数配合友元类外实现
    //需要加上一个空模板
    friend void printBase1<>(Base<T>& b);
};

int main()
{
    Base<int> b;
    printBase(b);
    printBase1(b);
    return 0;
}
```

9.异常处理

异常是程序在**执行期间**产生的问题。C++ 异常是指在程序运行时发生的特殊情况，比如尝试除以零的操作。

- **throw:** 当问题出现时，程序会抛出一个异常。这是通过使用 **throw** 关键字来完成的。
- **catch:** 在您想要处理问题的地方，通过异常处理程序捕获异常。**catch** 关键字用于捕获异常。
- **try: try** 块中的代码标识将被激活的特定异常。它后面通常跟着一个或多个 catch 块。

  下面是一个实例，抛出一个除以零的异常，并在 catch 块中捕获该异常

```
#include <iostream>
```

```
using namespace std;

double division(int a, int b)
{
   if( b == 0 )
   {
      throw b;
   }
   return (a/b);
}

int main ()
{
   int x = 50;
   int y = 0;
   double z = 0;

   try
   {
     z = division(x, y);
     cout << z << endl;
   }catch (int)//捕获的异常类型要和抛出的异常类型一样
   {
     cerr << "error" << endl;
   }

   return 0;
}
```

STL

STL广义上分为容器，算法，迭代器，容器和算法之间通过迭代器无缝连接

容器分为**序列式容器**和**关联式容器**

序列式容器:强调值的排序，序列式容器中的每个元素均有固定的位置

关联式容器:二叉树结构，各元素之间没有严格的物理上的顺序关系

算法:问题之解法也
有限的步骤，解决逻辑或数学上的问题，这一门学科我们叫做算法(**Algorithms**)算法分为:**质变算法**和**非质变算法。**

质变算法:是指运算过程中会更改区间内的元素的内容。例如拷贝，替换，制除等等

非质变算法:是指运算过程中不会更改区间内的元素内容，例如查找、计数、历、寻找极值等等

迭代器:容器和算法之间粘合剂
提供一种方法，使之能够依序寻访某个容器所含的各个元素，而又无需暴露该容器的内部表示方式。每个容器都有自己专属的迭代器种类

0. auto自动推导类型

c++11引用

```
    auto a = 1;//自动推导为int,auto定义的变量必须要初始值
    auto b = a;
    //可以遍历容器或其他序列的元素
    vector<int>v;
```

```
    v.push_back(1);
    v.push_back(2);
    v.push_back(3);
    for (auto& i : v)
    {
        i = i * 2;
    }
    for (auto it = v.begin(); it != v.end(); ++it)
    {
        auto& i = *it;
        i = i * 2;
    }
    for (auto i : v)
    {
        cout << i << endl;
    }
    string str = "abc";
    for (auto& i : str)
    {
        i = i + 1;
        cout << i << endl;
    }
```

1. vector

1.1 vector存放内置数据类型

容器：`vector`

算法：`for_each`

迭代器：`vector<int>::iterator`

```
#include<iostream>
#include<algorithm>
#include<vector>
using namespace std;

void val(int x)
{
    cout << x << endl;
}

int main()
{
    vector<int> v;
    v.push_back(10);
    v.push_back(20);
    v.push_back(30);

    //通过迭代器访问容器内的数据
    vector<int>::iterator itBegin = v.begin();//初始迭代器指向第一个元素
    vector<int>::iterator itEnd = v.end();//初始迭代器指向最后一个元素的下一位

    //第一种遍历方式
    while (itBegin != itEnd)
    {
        cout << *itBegin << endl;
```

```
        itBegin++;
    }

    //第二种遍历方式
    for (vector<int>::iterator it = v.begin(); it != v.end(); it++)
    {
        cout << *it << endl;
    }

    //第三种遍历方式
    for_each(v.begin(), v.end(), val);
    return 0;
}
```

1.2 vector容器中存放自定义数据类型

```
#include<iostream>
#include<algorithm>
#include<vector>
#include<string>
using namespace std;

class Person
{
public:
    string name;
    int age;

    Person(string name, int age)
    {
        this->name = name;
        this->age = age;
    }
};

int main()
{
    vector<Person> v;
    Person p1("aaa", 10);
    Person p2("bbb", 20);
    Person p3("ccc", 30);
    v.push_back(p1);
    v.push_back(p2);
    v.push_back(p3);

    for (vector<Person>::iterator it = v.begin(); it != v.end(); it++)
    {
        cout << "name" << (*it).name << "age" << (*it).age << endl;
        cout << "name" << it->name << "age" << it->age << endl;
    }

    vector<Person*> v1;
    Person p4("aaa", 10);
    Person p5("bbb", 20);
    Person p6("ccc", 30);
    v1.push_back(&p4);
    v1.push_back(&p5);
```

```
    v1.push_back(&p6);

    for (vector<Person*>::iterator it = v1.begin(); it != v1.end(); it++)
    {
        cout << "name" << (*it)->name << "age" << (*it)->age << endl;
    }
}
```

1.3容器嵌套容器

```
#include<iostream>
#include<algorithm>
#include<vector>
#include<string>
using namespace std;

int main()
{
    vector<vector<int>>v;
    vector<int>v1;
    vector<int>v2;
    vector<int>v3;
    vector<int>v4;

    for (int i = 0; i < 4; ++i)
    {
        v1.push_back(i + 1);
        v2.push_back(i + 2);
        v3.push_back(i + 3);
        v4.push_back(i + 4);
    }

    v.push_back(v1);
    v.push_back(v2);
    v.push_back(v3);
    v.push_back(v4);

    for (vector<vector<int>>::iterator it = v.begin(); it != v.end(); it++)
    {
        for (vector<int>::iterator vit = (*it).begin(); vit != (*it).end();
vit++)
        {
            cout << *vit << " ";
        }
        cout << endl;
    }

    return 0;
}
```

1.4操作

- vector的迭代器支持随机访问

头文件`vector`

- 构造操作

```
#include<iostream>
#include<vector>
using namespace std;

int main()
{
    //容器构造
    //1.默认构造
    vector<int>v1;
    for (int i = 0; i < 4; ++i) v1.push_back(i);
    //2.区间方法进行构造
    vector<int>v2(v1.begin(), v1.end());
    //3.给v3中加入10个200
    vector<int>v3(10, 200);
    //4.拷贝构造
    vector<int>v4(v1);
    return 0;
}
```

- 赋值操作

```
    vector<int>v1;
    for (int i = 0; i < 4; ++i) v1.push_back(i);

    vector<int>v2;
    v2 = v1;

    //assign
    vector<int>v3;
    v3.assign(v1.begin(), v1.end());

    vector<int>v4;
    v4.assign(10, 100);
```

- 容量和大小

```
vector<int>v1;
for (int i = 0; i < 4; ++i) v1.push_back(i);

if (v1.empty() == true)
{
    cout << "empty" << endl;
}
else
{
    cout << v1.capacity() << endl;//v1的容量
    cout << v1.size() << endl;//v1的大小
}
//设置v1长度若设置大于当前长度用0来填充，若小于则将后面都删除
v1.resize(15);
v1.resize(1);
```

- 插入和删除

```
    vector<int>v1;
```

```
    //尾插
    v1.push_back(10);
    v1.push_back(20);
    v1.push_back(30);
    //尾删
    v1.pop_back();
    //插入
    v1.insert(v1.begin(), 10);//最前面插入一个10
    v1.insert(v1.begin(), 5, 100);//最前面加入5个100
    //删除
    v1.erase(v1.begin());
    //清空
    v1.erase(v1.begin(), v1.end());
    v1.clear();
```

- 数据存取

```
vector<int>v1;
for (int i = 0; i < 10; ++i)
{
    v1.push_back(i);
    cout << v1[i] << endl;
    cout << v1.at(i) << endl;
}
cout << v1.front() << endl;//最前面的元素
cout << v1.back() << endl;//最后面的元素
```

- 容器互换

`v1.swap(v2);`交换容器

巧用swap可以收缩空间

`vector<int>(v).swap(v1);`后面v会被自动销毁

- 预留空间

```
vector<int>v1;
int num = 0;
int* p = NULL;
for (int i = 0; i < 10000; ++i)
{
    v1.push_back(i);
    if (p != &v1[0])
    {
        p = &v1[0];
        num++;//统计开辟次数,每次插满需要再次插入就会重新开辟新地址
    }
}
cout << num << endl;//24
vector<int>v2;
v2.reserve(10000);//预留一块空间;
num = 0;
p = NULL;
for (int i = 0; i < 10000; ++i)
{
    v2.push_back(i);
```

```
    if (p != &v2[0])
    {
        p = &v2[0];
        num++;//统计开辟次数,每次插满需要再次插入就会重新开辟新地址
    }
}
cout << num << endl;//1
```

2. deque

头文件：`#include<deque>`

双端数组，可以对头端进行插入删除操作

与vector区别：

vector对于头部的插入删除效率低，数据量越大，效率越低

deque对头部的插入删除速度比vector快

vector访问元素更快

2.1构造函数

```
    deque<int>d1;
    for (int i = 0; i < 10; ++i)
    {
        d1.push_back(i);
    }
    deque<int>d2(d1.begin(), d1.end());
    deque<int>d3(10, 100);
    deque<int>d4(d3);
```

2.2赋值操作

```
    deque<int>d1;
    for (int i = 0; i < 10; ++i)
    {
        d1.push_back(i);
    }
    deque<int>d2;
    d2 = d1;
    deque<int>d3;
    d3.assign(d1.begin(), d1.end());
    deque<int>d4;
    d4.assign(10, 100);
```

2.3大小操作

```
    deque<int>d1;
    for (int i = 0; i < 10; ++i)
    {
        d1.push_back(i);
    }
    if (d1.empty())
    {
        cout << "true" << endl;
    }
```

```
    else
    {
        cout << "false" << endl;
        cout << "size:" << d1.size() << endl;
    }
    d1.resize(16);//指定长度，没赋值的地方默认为0
    d1.resize(16, 1);//给超出地方赋值为1
    d1.resize(5);
```

2.4插入和删除

```
    deque<int>d1;
    //尾插
    d1.push_back(10);
    //头插
    d1.push_front(20);
    //尾删
    d1.pop_back();
    //头删
    d1.pop_front();
    //指定位置插入
    d1.insert(d1.begin(), 10);
    d1.insert(d1.begin(), 2, 10);//头部插入2个10
    deque<int>d2;
    d2.insert(d2.begin(), d1.begin(), d1.end());//头部插入d1头部到d1尾部的数据
    d2.erase(d2.begin()+1);//删除头部下一位数据，返回下一位的下一个数据位置
    d2.erase(d2.begin(), d2.end());//删除区间内的数据返回下一个数据的位置
    d2.clear();//清空所有数据
```

2.5数据存储

```
    deque<int>d1;
    d1.push_back(1);
    d1.push_back(2);
    for (int i = 0; i < d1.size(); ++i)
    {
        cout << d1[i] << endl;
        cout << d1.at(i) << endl;
    }
    d1.front();
    d1.back();
```

2.6排序

对于支持随机访问的迭代器的容器都可以利用sort算法来进行排序

```
    deque<int>d1;
    for (int i = 0; i < 10; ++i)
    {
        d1.push_back(10 - i);
    }
    sort(d1.begin(), d1.end());//默认从小到大
```

3. stack

后进先出

头文件`stack`

```
#include<iostream>
#include<stack>
using namespace std;

int main()
{
    stack<int>s;
    //入栈
    s.push(10);
    s.push(20);
    while (!s.empty())
    {
        cout << s.top() << endl;//打印栈顶元素
        s.pop();//出栈
    }
    cout << s.size() << endl;//栈的大小
    return 0;
}
```

4.queue

先进先出

```
#include<iostream>
#include<queue>
using namespace std;

class Person
{
public:
    Person(int a,int b)
    {
        m_a = a;
        m_b = b;
    }
    int m_a;
    int m_b;
};

int main()
{
    queue<Person>q;
    Person p1(10, 20);
    Person p2(20, 30);
    q.push(p1);
    q.push(p2);
    while (!q.empty())
    {
        cout << q.front().m_a << endl;//队头
        cout << q.back().m_a << endl;//队尾
    }
```

```
    cout << q.size() << endl;
    return 0;
}
```

5.list

STL中的链表是一个双向循环链表

采用动态存储分配，不会造成内存浪费和溢出

可以对任何位置快速的插入或删除元素，插入和删除都不会造成原有list迭代器的失效

容器遍历速度比数组慢，占用空间比数组大

5.1构造函数

```
#include<iostream>
#include<list>
using namespace std;

int main()
{
    list<int>l1;
    l1.push_back(10);
    l1.push_back(20);
    l1.push_back(30);
    l1.push_back(40);
    list<int>l2(l1.begin(), l1.end());
    list<int>l3(10, 1000);
    list<int>l4(l3);
    for (auto i : l1)
    {
        cout << i << endl;
    }
    return 0;
}
```

5.2赋值和交换

```
    //赋值
    list<int>l1;
    l1.push_back(10);
    l1.push_back(20);
    l1.push_back(30);
    l1.push_back(40);
    for (auto i : l1)
    {
        cout << i << endl;
    }
    list<int>l2;
    l2 = l1;
    list<int>l3;
    l3.assign(l2.begin(), l2.end());
    list<int>l4;
    l4.assign(10, 1000);

    //交换
    l1.swap(l2);
```

5.3大小操作

```
    if (l1.empty())
    {
        cout << "true" << endl;
    }
    else
    {
        cout << "false" << endl;
    }
    l1.resize(10, 10000);
    l1.resize(2);
```

5.4插入,删除,数据存取

```
    //头插
    l1.push_front(1);
    //尾删
    l1.pop_back();
    //头删
    l1.pop_front();
    //插入
    l1.insert(++l1.begin(), 100);
    //删除
    l1.erase(l1.begin());
    //移除，删除所有匹配的值
    l1.remove(30);//删除所有30
    l1.clear();
    //迭代器不支持随机访问
    l1.front();
    l1.back();
```

5.5反转和排序

```
#include<iostream>
#include<list>
using namespace std;

bool cmp(int v1, int v2)
{
    return v1 > v2;
}

int main()
{
    //赋值
    list<int>l1;
    l1.push_back(10);
    l1.push_back(20);
    l1.push_back(30);
    l1.push_back(30);
    //反转
    l1.reverse();
    //排序,不支持随机访问的迭代器的容器不能用快排
    //这些容器内部会对于一些算法
```

```
    l1.sort();//升序,不需要#include<algorithm>
    l1.sort(cmp);//降序
    return 0;
}
```